Engineering Handbook – Volume 02

System Architecture (Version 1.0)

Purpose

This volume defines the logical and physical architecture of the Energy Management Industrial IoT SaaS platform. It explains every major subsystem, why it exists, and how information flows through the platform.

Architecture Principles

The platform is API-first, Docker-first, multi-tenant, configuration-driven, modular, horizontally scalable and cloud-native. Business logic resides in the backend; edge devices collect telemetry only.

High-Level Architecture

Factory Equipment → Sensors/VFDs/PLCs → RS485 Modbus → Raspberry Pi Gateway → FastAPI Backend → PostgreSQL → Analytics → Dashboards → Mobile Apps.

Edge Layer

The Raspberry Pi acts as an edge gateway. It polls Modbus devices, buffers data during outages, timestamps readings, validates payloads and forwards telemetry securely to the cloud API.

Communication Layer

Initial communication uses Modbus RTU over RS485. MQTT will become the primary transport between gateways and the backend. HTTPS REST APIs handle configuration and administration.

Backend Layer

FastAPI exposes REST APIs, validates requests, authenticates users, applies business rules and persists data. Background workers will later process alarms, aggregation and notifications.

Database Layer

PostgreSQL stores tenants, users, departments, machine templates, machine instances, components, telemetry definitions and historical readings. Time-series optimization will be introduced later.

Multi-Tenant Model

Every resource belongs to a Company. Companies own Departments; Departments own Machines; Machines own Component Instances; Components produce Telemetry. Tenant isolation is mandatory.

Telemetry Pipeline

Device → Poller → Gateway → API → Validation → Database → Aggregation → Dashboard → Alarm Engine → Notification.

Future Services

Redis, Celery, MQTT Broker, Grafana, Flutter mobile apps, OCR services, predictive maintenance models and reporting services will be added without changing the core architecture.

Security

JWT authentication, role-based access control, TLS, audit logging, input validation, API versioning and secure secrets management are design requirements.

Deployment

Current deployment uses Docker containers on Ubuntu hosted in DigitalOcean with Portainer. Future deployments should support staging, production and automated CI/CD.

Design Decisions

FastAPI chosen for performance and type safety; PostgreSQL for relational integrity; Docker for portability; GitHub for version control; documentation maintained alongside code.

Logical Component Diagram

Company
└── Department
  └── Machine
    └── Machine Component
      └── Tag Definition
        └── Telemetry Data

Future volumes will expand every subsystem with sequence diagrams, database diagrams, deployment topology, API contracts and implementation standards.